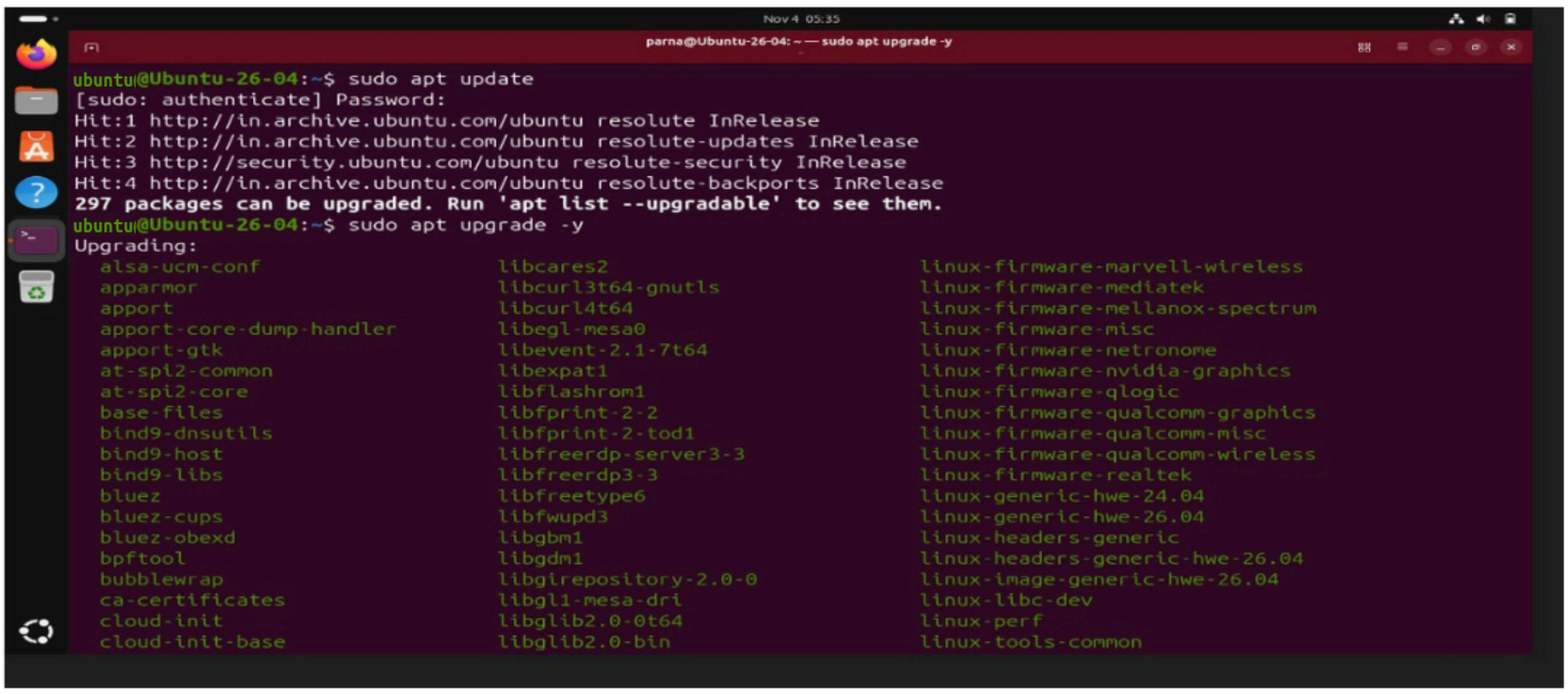
LAB 4: SDN

1. Update Ubuntu

sudo apt update && sudo apt upgrade -y



Updates the Ubuntu system.

2.Check Mininet installation

mn --version

```
Processing triggers for man-db (2.13.1-1build1)…
ubuntu@Ubuntu-26-04:~$ mn --version
2.3.0
```

Checks Mininet for creating virtual networks.

3. Install Open vSwitch

sudo apt install openvswitch-switch -y

```
ubuntu@Ubuntu-26-04:~$ sudo apt install openvswitch-switch -y
openvswitch-switch is already the newest version (3.7.1-2).
openvswitch-switch set to manually installed.
The following packages were automatically installed and are no longer required:
  linux-headers-7.0.0-29          linux-main-modules-zfs-7.0.0-29-generic  linux-tools-7.0.0-29-generic
  linux-headers-7.0.0-29-generic  linux-modules-7.0.0-29-generic
  linux-image-7.0.0-29-generic    linux-tools-7.0.0-29
Use 'sudo apt autoremove' to remove them.

Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 3
ubuntu@Ubuntu-26-04:~$
```

Installs the software-based SDN switch.

4. Check Open vSwitch

sudo systemctl status openvswitch-switch

```
ubuntu@Ubuntu-26-04:~$ sudo systemctl status openvswitch-switch
● openvswitch-switch.service - Open vSwitch
     Loaded: loaded (/usr/lib/systemd/system/openvswitch-switch.service; enabled; preset: enabled)
     Active: active (exited) since Wed 2026-11-04 15:19:50 UTC; 10min ago
 Invocation: ecd8cbb3e63545cf8b1138ec6156e34b
    Process: 1605 ExecStart=/bin/true (code=exited, status=0/SUCCESS)
   Main PID: 1605 (code=exited, status=0/SUCCESS)
   Mem peak: 1.7M
        CPU: 34ms

Nov 04 15:19:50 Ubuntu-26-04 systemd[1]: Starting openvswitch-switch.service - Open vSwitch...
Nov 04 15:19:50 Ubuntu-26-04 systemd[1]: Finished openvswitch-switch.service - Open vSwitch.
ubuntu@Ubuntu-26-04:~$
```

Checks whether Open vSwitch is running.

5. Test Mininet

sudo mn

This starts a basic Mininet network.

mininet> nodes

It displays the available virtual hosts and switch.

mininet> pingall

Tests connectivity between the virtual hosts.

mininet> exit

Exits Mininet.

```
ubuntu@Ubuntu-26-04:~$ sudo mn
*** No default OpenFlow controller found for default switch!
*** Falling back to OVS Bridge
*** Creating network
*** Adding controller
*** Adding hosts:
h1 h2
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1)
*** Configuring hosts
h1 h2
*** Starting controller

*** Starting 1 switches
s1 ...
*** Starting CLI:
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2
h2 -> h1
*** Results: 0% dropped (2/2 received)
mininet> exit
*** Stopping 0 controllers

*** Stopping 2 links
```

6. Install Python and Virtual Environment

sudo apt install python3-pip python3-venv -y

```
parna@Ubuntu-26-04:~$ sudo apt install python3-pip python3-venv -y
The following packages were automatically installed and are no longer required:
  linux-headers-7.0.0-29          linux-main-modules-zfs-7.0.0-29-generic  linux-tools-7.0.0-29-generic
  linux-headers-7.0.0-29-generic  linux-modules-7.0.0-29-generic
  linux-image-7.0.0-29-generic    linux-tools-7.0.0-29
Use 'sudo apt autoremove' to remove them.

Installing:
  python3-pip  python3-venv

Installing dependencies:
  libexpat1-dev   libpython3.14-dev  python3-pip-whl         python3-wheel   python3.14-venv
  libpython3-dev  python3-dev        python3-setuptools-whl  python3.14-dev  zlib1g-dev

Summary:
  Upgrading: 0, Installing: 12, Removing: 0, Not Upgrading: 3
  Download size: 11.6 MB
  Space needed: 45.2 MB / 14.2 GB available

Get:1 http://in.archive.ubuntu.com/ubuntu resolute-updates/main amd64 libexpat1-dev amd64 2.7.4-1ubuntu0.2 [149 kB]
Get:2 http://in.archive.ubuntu.com/ubuntu resolute-updates/main amd64 zlib1g-dev amd64 1:1.3.dfsg+really1.3.1-1ubuntu
3.2 [895 kB]
Get:3 http://in.archive.ubuntu.com/ubuntu resolute-updates/main amd64 libpython3.14-dev amd64 3.14.4-1ubuntu0.2 [5,98
6 kB]
Get:4 http://in.archive.ubuntu.com/ubuntu resolute/main amd64 libpython3-dev amd64 3.14.3-0ubuntu2 [8,736 B]
Get:5 http://in.archive.ubuntu.com/ubuntu resolute-updates/main amd64 python3.14-dev amd64 3.14.4-1ubuntu0.2 [510 kB]
Get:6 http://in.archive.ubuntu.com/ubuntu resolute/main amd64 python3-dev amd64 3.14.3-0ubuntu2 [26.6 kB]
Get:7 http://in.archive.ubuntu.com/ubuntu resolute/universe amd64 python3-wheel all 0.46.3-2 [27.9 kB]
```

Installs Python package and virtual-environment support.

7. Create Ryu Environment

python3 -m venv ryu-env

source ryu-env/bin/activate

```
parna@Ubuntu-26-04:~$ python3 -m venv ryu-env
parna@Ubuntu-26-04:~$ source ryu-env/bin/activate
```

Creates and activates a Python environment for Ryu.

8. Install Ryu

pip install ryu

```
(ryu39) parna@Ubuntu-26-04:~$ source ~/ryu-env/bin/activate
(ryu-env) parna@Ubuntu-26-04:~$ uv pip install --python ~/ryu39/bin/python setuptools==59.6.0 wheel pbr eventlet==0.3
0.2
Using Python 3.9.25 environment at: ryu39
Resolved 8 packages in 1.01s
Prepared 1 package in 439ms
Installed 8 packages in 181ms
 + dnspython==1.16.0
 + eventlet==0.30.2
 + greenlet==3.2.5
 + packaging==26.3
 + pbr==7.1.3
 + setuptools==59.6.0
 + six==1.17.0
 + wheel==0.48.0
(ryu-env) parna@Ubuntu-26-04:~$ uv pip install --no-build-isolation --python ~/ryu39/bin/python ryu==4.34
Using Python 3.9.25 environment at: ryu39
Resolved 25 packages in 1.64s
Installed 10 packages in 352ms
Prepared 9 packages without build isolation in 1.20s
Installed 9 packages in 26ms
 + certifi==2026.10.23
 + charset-normalizer==3.5.2
 + idna==3.20
 + msgpack==1.1.2
 + netaddr==1.3.0
 + oslo-config==10.0.0
```

Installs the Ryu SDN controller.

9. Test Ryu Installation

ryu-manager –version

```
(ryu-env) ubuntu@Ubuntu-26-04:~$ ~/ryu39/bin/ryu-manager --version
/home/parna/ryu39/lib/python3.9/site-packages/ryu/lib/packet/cfm.py:271: SyntaxWarning: "is not" with a literal. Did
you mean "!="?
  assert interval is not 0
ryu-manager 4.34
```

Displays the installed Ryu version.

10. Start Ryu Controller

ryu-manager ryu.app.simple_switch_13

```
(ryu39) ubuntu@Ubuntu-26-04:~$ ryu-manager ryu.app.simple_switch_13
loading app ryu.app.simple_switch_13
loading app ryu.controller.ofp_handler
instantiating app ryu.app.simple_switch_13 of SimpleSwitch13
instantiating app ryu.controller.ofp_handler of OFPHandler
packet in 1 7a:56:c5:28:39:7f 33:33:00:00:00:16 2
packet in 1 7a:56:c5:28:39:7f 33:33:00:00:00:02 2
packet in 1 8e:97:cf:47:2d:c4 33:33:00:00:00:16 1
packet in 1 8e:97:cf:47:2d:c4 33:33:00:00:00:02 1
packet in 1 7a:56:c5:28:39:7f 33:33:00:00:00:16 2
packet in 1 52:c6:29:ea:0f:59 33:33:00:00:00:16 3
packet in 1 52:c6:29:ea:0f:59 33:33:00:00:00:02 3
packet in 1 8e:97:cf:47:2d:c4 33:33:00:00:00:16 1
packet in 1 52:c6:29:ea:0f:59 33:33:00:00:00:16 3
packet in 1 7a:56:c5:28:39:7f 33:33:00:00:00:02 2
packet in 1 52:c6:29:ea:0f:59 33:33:00:00:00:02 3
packet in 1 8e:97:cf:47:2d:c4 33:33:00:00:00:02 1
packet in 1 52:c6:29:ea:0f:59 33:33:00:00:00:02 3
packet in 1 8e:97:cf:47:2d:c4 33:33:00:00:00:02 1
packet in 1 7a:56:c5:28:39:7f 33:33:00:00:00:02 2
packet in 1 52:c6:29:ea:0f:59 33:33:00:00:00:02 3
packet in 1 8e:97:cf:47:2d:c4 33:33:00:00:00:02 1
packet in 1 7a:56:c5:28:39:7f 33:33:00:00:00:02 2
packet in 1 52:c6:29:ea:0f:59 33:33:00:00:00:02 3
packet in 1 8e:97:cf:47:2d:c4 33:33:00:00:00:02 1
packet in 1 7a:56:c5:28:39:7f 33:33:00:00:00:02 2
packet in 1 52:c6:29:ea:0f:59 33:33:00:00:00:02 3
```

Starts Ryu with a basic OpenFlow switching application.

Keep this terminal open.

11. Create Mininet SDN Topology

Open a new terminal:

sudo mn --topo single,3 --controller=remote --switch ovsk

```
ubuntu@Ubuntu-26-04:~$ sudo mn --topo single,3 --controller=remote --switch ovsk
[sudo: authenticate] Password:
*** Creating network
*** Adding controller
Connecting to remote controller at 127.0.0.1:6653
*** Adding hosts:
h1 h2 h3
*** Adding switches:
s1
*** Adding links:
(h1, s1) (h2, s1) (h3, s1)
*** Configuring hosts
h1 h2 h3
*** Starting controller
c0
*** Starting 1 switches
s1 ...
*** Starting CLI:
```

Creates 3 virtual hosts connected to an Open vSwitch and remote Ryu controller.

12. Check Nodes

mininet> nodes

```
mininet> nodes
available nodes are:
c0 h1 h2 h3 s1
mininet>
```

Displays H1, H2, H3 and S1.

13. Check Connections

mininet> net

```
mininet> net
h1 h1-eth0:s1-eth1
h2 h2-eth0:s1-eth2
h3 h3-eth0:s1-eth3
s1 lo:  s1-eth1:h1-eth0 s1-eth2:h2-eth0 s1-eth3:h3-eth0
c0
```

Displays the connections between hosts and the switch.

14. Test Connectivity

mininet> pingall

Verifies communication between all virtual hosts.

mininet> h1 ping h2

Tests connectivity between H1 and H2.

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3
h2 -> h1 h3
h3 -> h1 h2
*** Results: 0% dropped (6/6 received)
mininet> h1 ping -c 4 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=1.71 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.129 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.128 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.103 ms

--- 10.0.0.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3091ms
rtt min/avg/max/mdev = 0.103/0.516/1.707/0.687 ms
```

15. View Flow Rules

Open another terminal:

sudo ovs-ofctl -O OpenFlow13 dump-flows s1

```
ubuntu@Ubuntu-26-04:~$ sudo ovs-ofctl -O OpenFlow13 dump-flows s1
[sudo: authenticate] Password:
 cookie=0x0, duration=155.979s, table=0, n_packets=9, n_bytes=714, priority=1,in_port="s1-eth2",dl_src=7a:56:c5:28:39
:7f,dl_dst=8e:97:cf:47:2d:c4 actions=output:"s1-eth1"
 cookie=0x0, duration=155.972s, table=0, n_packets=8, n_bytes=616, priority=1,in_port="s1-eth1",dl_src=8e:97:cf:47:2d
:c4,dl_dst=7a:56:c5:28:39:7f actions=output:"s1-eth2"
 cookie=0x0, duration=155.948s, table=0, n_packets=8, n_bytes=672, priority=1,in_port="s1-eth3",dl_src=52:c6:29:ea:0f
:59,dl_dst=8e:97:cf:47:2d:c4 actions=output:"s1-eth1"
 cookie=0x0, duration=155.943s, table=0, n_packets=7, n_bytes=574, priority=1,in_port="s1-eth1",dl_src=8e:97:cf:47:2d
:c4,dl_dst=52:c6:29:ea:0f:59 actions=output:"s1-eth3"
 cookie=0x0, duration=155.908s, table=0, n_packets=3, n_bytes=238, priority=1,in_port="s1-eth3",dl_src=52:c6:29:ea:0f
:59,dl_dst=7a:56:c5:28:39:7f actions=output:"s1-eth2"
 cookie=0x0, duration=155.901s, table=0, n_packets=2, n_bytes=140, priority=1,in_port="s1-eth2",dl_src=7a:56:c5:28:39
:7f,dl_dst=52:c6:29:ea:0f:59 actions=output:"s1-eth3"
 cookie=0x0, duration=328.967s, table=0, n_packets=36, n_bytes=2556, priority=0 actions=CONTROLLER:65535
```

Displays the flow rules installed in the Open vSwitch.

16. Add a Simple Filtering Rule

sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=100,in_port=1,actions=drop"

```
ubuntu@Ubuntu-26-04:~$ sudo ovs-ofctl -O OpenFlow13 add-flow s1 "priority=100,in_port=1,actions=drop"
```

Drops packets entering S1 through port 1.

17. Test the Filtering Rule

mininet> h1 ping h2

```
mininet> h1 ping -c 4 h3
PING 10.0.0.3 (10.0.0.3) 56(84) bytes of data.
64 bytes from 10.0.0.3: icmp_seq=1 ttl=64 time=1.79 ms
64 bytes from 10.0.0.3: icmp_seq=2 ttl=64 time=0.543 ms
64 bytes from 10.0.0.3: icmp_seq=3 ttl=64 time=0.094 ms
64 bytes from 10.0.0.3: icmp_seq=4 ttl=64 time=0.090 ms

--- 10.0.0.3 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3051ms
rtt min/avg/max/mdev = 0.090/0.630/1.794/0.696 ms
mininet> h1 ping -c 4 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.

--- 10.0.0.2 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3111ms
```

Checks the effect of the filtering rule on H1's traffic.

18. Remove Flow Rules

sudo ovs-ofctl -O OpenFlow13 del-flows s1

```
ubuntu@Ubuntu-26-04:~$ sudo ovs-ofctl -O OpenFlow13 del-flows s1
```

Removes the manually added flow rules.

19. Clean Mininet

sudo mn -c

```
mininet> h1 ping -c 4 h2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=15.2 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=4.61 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=0.128 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.120 ms

--- 10.0.0.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3032ms
rtt min/avg/max/mdev = 0.120/5.024/15.234/6.173 ms
mininet> sudo mn -c
```

Cleans up Mininet interfaces and processes.

Result: The SDN environment was successfully set up using Mininet, Open vSwitch, and the Ryu Controller. Basic network traffic forwarding and filtering were also implemented and tested successfully.